

BlinkIT Grocery Sales Analytics — Solution Design Document

1. Purpose

This document defines the technical solution for the BlinkIT Grocery Sales Analytics Power BI dashboard.

It describes the architecture, data preparation, semantic model, DAX measures, dashboard behavior, validation, deployment, governance, and scalability strategy.

2. Solution Summary

The current solution follows:

Excel source → Power Query / Power BI data preparation → Power BI semantic model → DAX measures → interactive dashboard

flowchart LR

A→[Excel Dataset]→B[Power Query / Data Preparation]

B --> C[Power BI Semantic Model]

C --> D[DAX Measures]

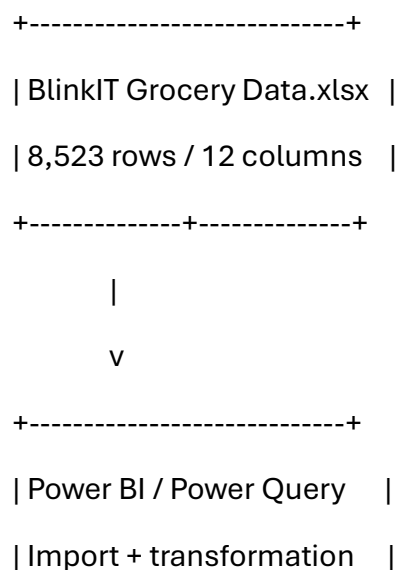
D --> E[Interactive Dashboard]

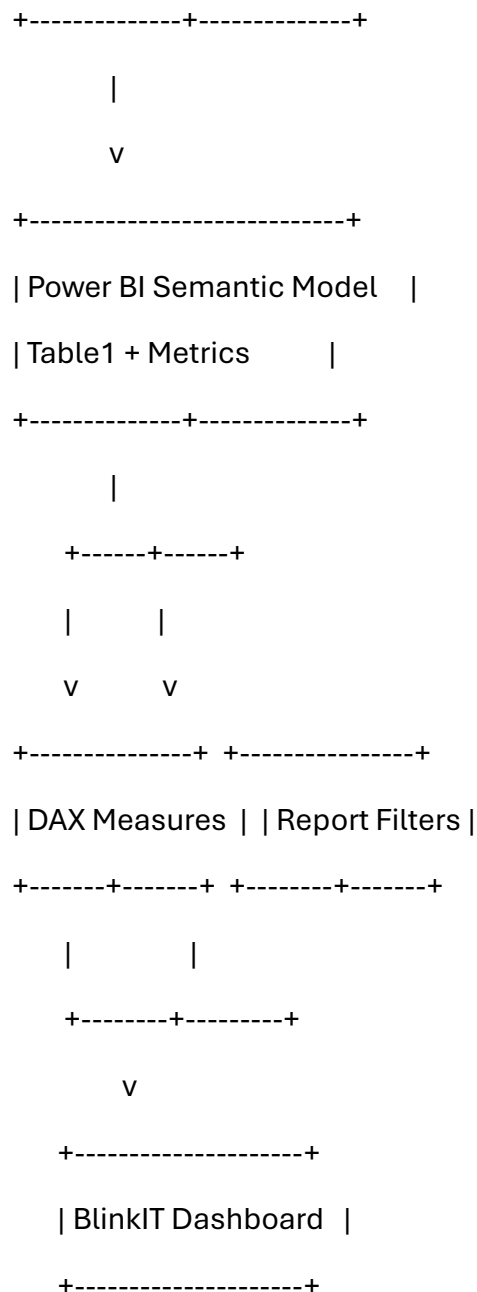
E --> F[Business Users]

The supplied PBIX contains a primary analytical table named Table1 and a metrics/support table named Metrics referenced by the report.

3. Architecture

Current Architecture





Target Production Architecture

flowchart LR

A[Source Systems] --> B[Data Lake / Warehouse]

B --> C[ETL / ELT]

C --> D[Curated Retail Model]

D --> E[Power BI Semantic Model]

E --> F[Power BI Report]

F --> G[Business Users]

The current Excel-based solution is appropriate for a portfolio project and small analytical extracts. The target architecture is recommended when refresh frequency or data volume increases.

4. Source System

Source File

BlinkIT Grocery Data.xlsx

Source Structure

Field	Analytical Role
Item Fat Content	Product attribute
Item Identifier	Product identifier
Item Type	Product category
Outlet Establishment Year	Outlet lifecycle attribute
Outlet Identifier	Outlet identifier
Outlet Location Type	Geographic tier
Outlet Size	Outlet dimension
Outlet Type	Outlet-format dimension
Item Visibility	Product visibility metric
Item Weight	Product attribute
Sales	Primary measure
Rating	Rating metric

5. Data Preparation

Recommended Power Query steps:

1. Import the Excel workbook.
2. Promote the correct header row.
3. Set data types.
4. Trim text columns.
5. Standardize categorical values.

6. Validate identifiers.
7. Profile null values.
8. Validate numeric ranges.
9. Remove accidental duplicates only when business rules confirm they are duplicates.
10. Load the curated table.

Recommended Data Types

Column	Type
Item Identifier	Text
Item Fat Content	Text
Item Type	Text
Outlet Identifier	Text
Outlet Location Type	Text
Outlet Size	Text
Outlet Type	Text
Outlet Establishment Year	Whole Number
Item Visibility	Decimal Number
Item Weight	Decimal Number
Sales	Decimal Number
Rating	Decimal Number

6. Data Model

Current Logical Model

- Table1 — primary analytical table.
- Metrics — metric-selection/support table.

Recommended Target Model

For future scalability, use a star schema:

flowchart TB

FI[Fact Sales]

DI[Dim Item]

DO[Dim Outlet]

DM[Dim Metric]

DI --> FI

DO --> FI

DM --> FI

Suggested responsibilities:

Fact Sales

- Sales
- Rating
- Item Visibility
- Item Weight
- Item Identifier
- Outlet Identifier

Dim Item

- Item Identifier
- Item Type
- Standardized Fat Content

Dim Outlet

- Outlet Identifier
- Location Type
- Size
- Outlet Type
- Establishment Year

Dim Metric

- Metric name used by the selector

The current flat-table model is adequate for the supplied dataset; a star schema becomes preferable as the project expands.

7. Data Standardization Rules

Normalize fat-content values:

LF → Low Fat

Low Fat → Low Fat

low fat → Low Fat

reg → Regular

Regular → Regular

Example Power Query pattern:

```
= Table.TransformColumns(  
    PreviousStep,  
    {  
        "Item Fat Content",  
        each  
            if Text.Lower(Text.Trim(_)) = "lf" or  
               Text.Lower(Text.Trim(_)) = "low fat"  
            then "Low Fat"  
            else if Text.Lower(Text.Trim(_)) = "reg" or  
                   Text.Lower(Text.Trim(_)) = "regular"  
            then "Regular"  
            else Text.Trim(_),  
        type text  
    }  
)
```

8. Business Logic

Total Sales

Total Sales =

SUM ('Table1'[Sales])

Total Outlets

Total Outlets =

DISTINCTCOUNT ('Table1'[Outlet Identifier])

Average Sales

Average Sales =

AVERAGE ('Table1'[Sales])

Average Rating

Average Rating =

AVERAGE ('Table1'[Rating])

These measures should be centralized in the semantic model instead of relying on repeated implicit aggregations.

9. Metric Selector

The PBIX contains a Metrics table used by a slicer/report interaction.

Recommended measure-switch pattern:

Selected Metric =

```
SWITCH (
    SELECTEDVALUE ( Metrics[Metrics] ),
    "Total Sales", [Total Sales],
    "Average Sales", [Average Sales],
    "Average Rating", [Average Rating],
    [Total Sales]
)
```

If the existing PBIX already uses another working implementation, retain it and use the above as the documented target pattern.

10. Report Filtering

The dashboard includes interactive filters for:

- Outlet Location Type.

- Outlet Size.
- Metric selection.

Expected behavior:

Outlet Location Type = Tier 3

should update all compatible KPIs, charts, and detail tables.

Likewise:

Outlet Size = Medium

should update all dependent calculations.

11. Visualization Design

Visual	Dimension	Metric / Purpose
KPI cards	—	Total Sales, Total Outlets, Average Sales, Average Rating
Location comparison	Outlet Location Type	Average Sales
Outlet-size visual	Outlet Size	Average Sales
Establishment-year visual	Establishment Year	Average Sales
Fat-content visual	Item Fat Content	Total Sales / Average Sales
Item-category visual	Item Type	Average Sales
Detail matrix	Item Type, Year, Size, Type	Average Sales, Total Sales, Average Rating

12. Performance Design

The supplied dataset contains only **8,523 rows**, so Power BI Import mode is appropriate.

Recommended practices:

- Prefer Import mode.
- Use explicit DAX measures.
- Avoid unnecessary calculated columns.
- Remove unused columns in production models.

- Normalize dimensions when the model grows.
- Avoid unnecessary high-cardinality calculated fields.
- Use Performance Analyzer when optimizing visuals.

For this dataset, inefficient modeling or excessive visual complexity is a greater risk than raw data volume.

13. Validation Strategy

Validate Power BI results against the supplied source:

Check	Expected
Row count	8,523
Unique outlets	10
Item categories	16
Total Sales	1,201,681.49
Average Sales	140.99
Average Rating	3.97

Filter Validation

For every slicer:

1. Select one value.
2. Confirm KPI changes.
3. Confirm intended visuals respond.
4. Clear the selection.
5. Confirm original totals return.

Aggregation Validation

For each chart, compare Power BI output with source-level group-by results.

14. Data Quality Tests

DQ-01 — Identifier Completeness

Item Identifier and Outlet Identifier should not be null.

DQ-02 — Sales Completeness

Sales should not be null.

DQ-03 — Rating Range

Validate ratings against the approved business range.

DQ-04 — Establishment Year

Validate establishment years against expected business ranges.

DQ-05 — Category Standardization

No unexpected fat-content variants should remain after transformation.

DQ-06 — Missing Weight

The source contains **1,463 missing Item Weight values**. This should be documented rather than silently replaced.

15. Security and Governance

The portfolio solution does not require row-level security because it is an aggregate analytical dashboard.

For enterprise deployment, consider:

- Row-Level Security by outlet/region.
- Workspace access controls.
- Certified semantic models.
- Sensitivity labels.
- Controlled refresh credentials.
- Separate development, test, and production workspaces.

16. Deployment Strategy

Development

1. Update source/query logic.
2. Validate transformations.
3. Validate measures.
4. Test dashboard interactions.

Test

1. Reconcile KPIs.
2. Test slicers.

3. Test visual interactions.
4. Validate formatting.
5. Check performance.

Production

1. Publish PBIX to Power BI Service.
2. Configure dataset credentials.
3. Configure refresh schedule.
4. Configure workspace permissions.
5. Publish/share the report.

17. Version Control Strategy

Power BI .pbix files are binary artifacts and are not ideal for line-by-line Git diffs.

Recommended repository structure:

powerbi/

 Blinkit.pbix

docs/

 PROJECT_DESIGN_DOCUMENT.md

 SOLUTION_DESIGN_DOCUMENT.md

screenshots/

 dashboard.png

For enterprise development, consider Power BI Project (.pbip) format where supported because it is more source-control-friendly.

18. Error Handling and Recovery

Source File Changes

- Verify sheet/table name.
- Verify column names.
- Re-run Power Query.

- Check data types.
- Reconcile row counts and KPI totals.

Schema Changes

- Update Power Query transformations.
- Update DAX measures.
- Update visual field mappings.
- Re-test dependent visuals.

Refresh Failures

1. Identify the failed query.
2. Validate source availability.
3. Check credentials.
4. Check schema changes.
5. Review Power Query errors.
6. Reconcile refreshed KPIs.

19. Observability

For a production implementation, track:

- Refresh success/failure.
- Refresh duration.
- Row counts.
- Null counts.
- KPI reconciliation.
- Query/visual performance.
- Data-source availability.

20. Scalability Roadmap

Phase 1 — Current Portfolio Solution

Excel → Power BI → Dashboard

Phase 2 — Improved BI Model

Excel/CSV → Power Query → Star Schema → Power BI

Phase 3 — Enterprise Analytics

Operational Sources



Data Lake / Warehouse



ETL / ELT



Curated Retail Model



Power BI Semantic Model



Reports / Executive Dashboards

Phase 4 — Advanced Analytics

Add:

- Forecasting.
- Product demand modeling.
- Promotion effectiveness.
- Margin optimization.
- Outlet benchmarking.
- Inventory analytics.
- Anomaly detection.

21. Technical Risks

Risk	Impact	Mitigation
Inconsistent category labels	Incorrect grouping	Standardize in Power Query
Missing item weights	Incomplete weight analysis	Document nulls
Binary PBIX versioning	Poor Git diffs	Prefer PBIP for future development

Excel schema changes	Refresh failure	Add schema validation
Implicit measures	Inconsistent business logic	Use explicit DAX measures
Increasing dataset size	Slower refresh/reporting	Move toward star schema/warehouse
Lack of transaction date	Limited trend analysis	Add transaction-level date in future source

22. Acceptance Criteria

The technical solution is accepted when:

- Core KPIs reconcile to the source.
- Slicers operate correctly.
- Visuals respond consistently to filter context.
- Data transformations are documented.
- Business measures have explicit definitions.
- Known data-quality issues are documented.
- The PBIX opens and refreshes successfully.
- Repository documentation explains both business and technical design.

23. Recommended GitHub Presentation

The repository README should lead with:

1. Project title.
2. One-line business objective.
3. Dashboard screenshot.
4. Key KPIs.
5. Business questions answered.
6. Tools and technologies.
7. Data/model overview.
8. Key insights.
9. Repository structure.

10. Documentation links.
11. How to open the PBIX.
12. Future enhancements.

24. Final Technical Recommendation

The existing Power BI solution is appropriate for a portfolio-scale retail analytics project. The highest-value improvements are:

1. Standardize Item Fat Content.
2. Replace implicit aggregations with documented explicit DAX measures.
3. Move toward a star schema if the model expands.
4. Add a dashboard screenshot and polished README.
5. Consider PBIP for future source-control-friendly Power BI development.
6. Add automated data-quality/reconciliation checks if the source becomes refreshable.